

Artificial Intelligence

Prof. Chang-Hee Han

4

Neural Network Basic: Classification and Regression

4.0 This chapter covers

▣ Getting Started with Neural Networks

- Goal: Learn how to use neural networks for real problems
- Three main tasks:
  - ➢ Movie review: positive or negative → **Binary Classification**
  - ➢ House price: predict value → **Scalar Regression**

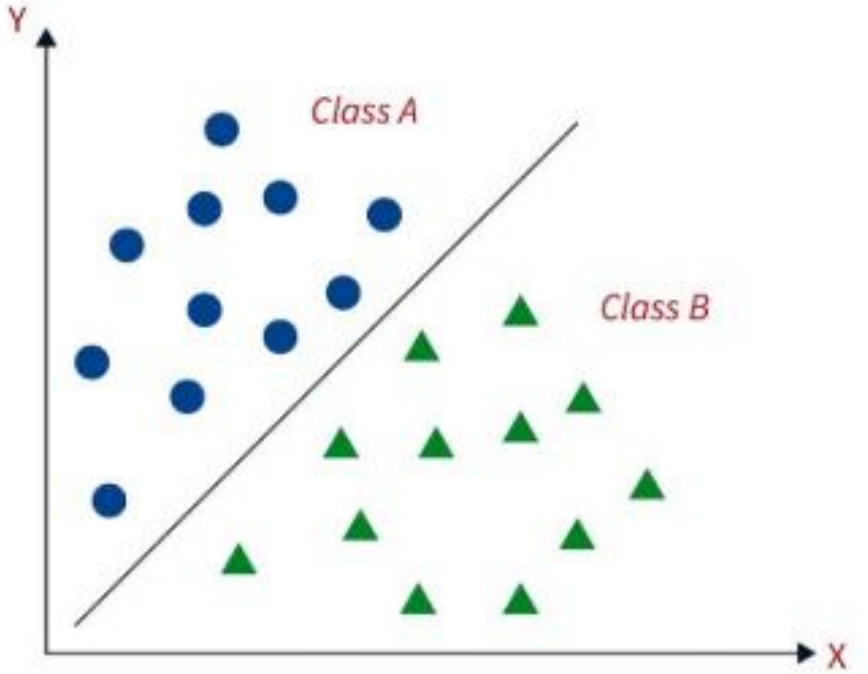
▣ What You Will Learn

- Data preprocessing
- Basic model design
- Model testing and evaluation

4.0 This chapter covers

▣ Classification vs Regression

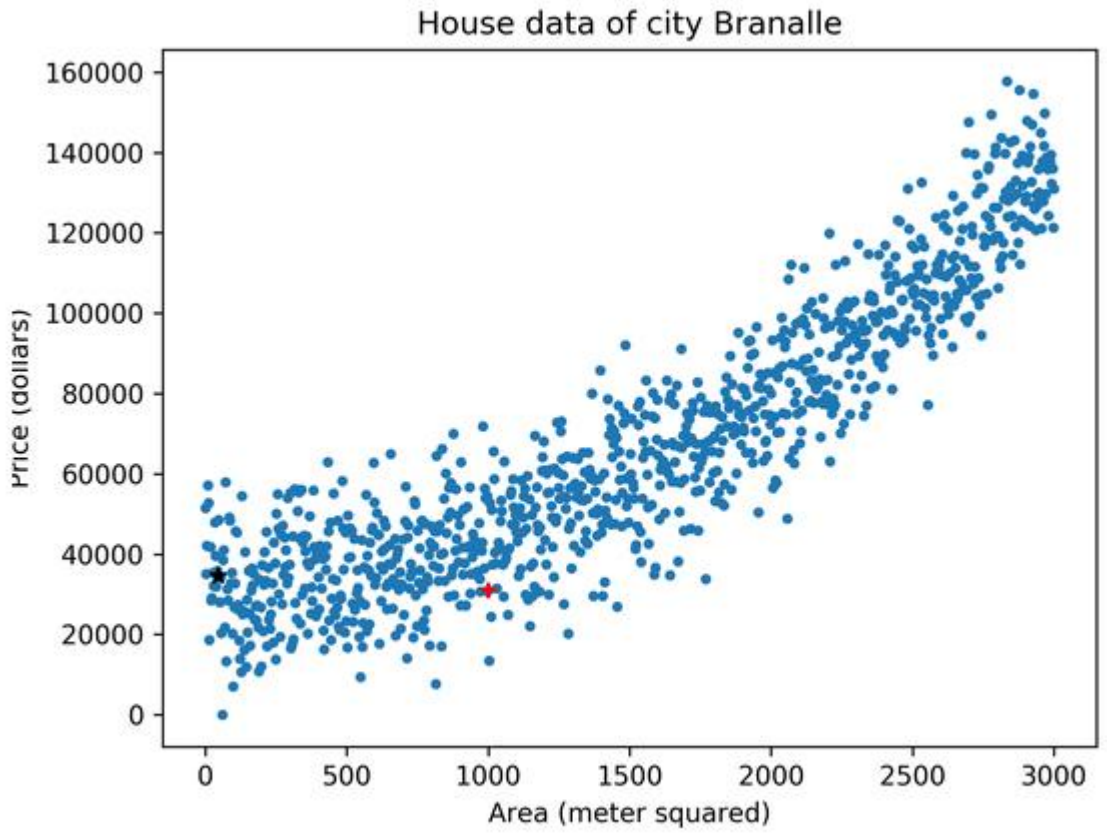
- Classification
  - ➢ Goal: Put data into groups (labels)
  - ➢ Output: Category / class
  - ➢ Examples:
    - Email: spam or not spam
    - Movie review: positive or negative



4.0 This chapter covers

▣ Classification vs Regression

- Regression
  - ➢ Goal: Predict a number (value)
  - ➢ Output: Continuous value
  - ➢ Examples:
    - House price prediction
    - Predicting temperature



4.1 Classifying movie reviews: A binary classification example

4.1.1 IMDB Dataset

- The IMDB dataset has 50,000 movie reviews from the Internet Movie Database.
- It is split into 25,000 training data and 25,000 test data.
- Each part has 50% negative and 50% positive reviews.
- Like the MNIST dataset, the IMDB dataset is also included in Keras.
- The data is already processed: each review (a word sequence) is changed into a number sequence.
- Each number shows a unique word from the dictionary.
- With this processed data, we can focus on building, training, and testing models.

4.1 Classifying movie reviews: A binary classification example

4.1.1 IMDB Dataset

- The code below loads the dataset (the first run will download about 17MB of data to the computer).

Listing 4.1 Loading the IMDB dataset

```
from tensorflow.keras.datasets import imdb
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000)
```

- The option num_words=10000 means: only use the 10,000 most common words in the training data.
- This makes a vector dataset with a good size.
- If we do not limit it, the training data has 88,585 unique words.
- This is too many and not useful.
- Many words appear in only one sample, so they are not helpful for classification.

4.1 Classifying movie reviews: A binary classification example

4.1.1 IMDB Dataset

- The arrays train_data and test_data contain the reviews.
- Each review is a list of word indexes (encoded word sequence).
- train_labels and test_labels are lists with 0 for negative and 1 for positive.

```
>>> train_data[0]
[1, 14, 22, 16, ... 178, 32]
>>> train_labels[0]
1
```

- Because we limit the words to the 10,000 most common, the word index does not go over 9,999.

```
>>> max([max(sequence) for sequence in train_data])
9999
```

4.1 Classifying movie reviews: A binary classification example

4.1.1 IMDB Dataset

- For fun, let's see how one review data can be changed back into the **original English words**.

Listing 4.2 Decoding reviews back to text

```
word_index = imdb.get_word_index()
reverse_word_index = dict(
    [(value, key) for (key, value) in word_index.items()])
decoded_review = " ".join(
    [reverse_word_index.get(i - 3, "?") for i in train_data[0]])
```

word_index is a dictionary mapping words to an integer index.

Reverses it, mapping integer indices to words

Decodes the review. Note that the indices are offset by 3 because 0, 1, and 2 are reserved indices for "padding," "start of sequence," and "unknown."

4.1 Classifying movie reviews: A binary classification example

4.1.2 Preparing the data

- We cannot give a list of numbers directly to a neural network.
- **Each list has a different length, but the network expects the same size batches.**
- There are two ways to change lists into tensors:
  - **Padding**: add padding so all lists have the same length → change to an integer tensor of shape *(samples, max_length)*.
    - Then start the network with a layer that can handle integers (**Embedding layer**).
  - **Multi-hot encoding**: change lists into **0/1 vectors**.
    - Example: sequence [8, 5] → index 8 and 5 are 1, all others are 0, in a 10,000-dimensional vector.
    - Then use a **Dense layer** as the first layer to work with these floating-point vectors.

4.1.2 Preparing the data

- Multi-hot encoding: change lists into 0/1 vectors.
  - This method is used to treat data as categories (qualitative variables), not as numbers.
  - With this encoding, the input data will always have the same shape when it goes into the Keras model layers.
  - A similar method is one-hot encoding
    - ✓ only one position has the number 1, all others are 0.
  - In multi-hot encoding, there can be more than one 1 in the vector.

4.1.2 Preparing the data

- Here, we will use the second method and, to help understanding, we will make the data into a multi-hot vector directly.

Listing 4.3 Encoding the integer sequences via multi-hot encoding

```
import numpy as np
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        for j in sequence:
            results[i, j] = 1.
    return results
x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)
```

Creates an all-zero matrix of shape (len(sequences), dimension)

Sets specific indices of results[i] to 1s

Vectorized training data

Vectorized test data

4.1 Classifying movie reviews: A binary classification example

4.1.2 Preparing the data

- Now the samples look like this.

```
>>> x_train[0]
array([ 0.,  1.,  1., ...,  0.,  0.,  0.])
```

- The labels can be easily changed into vectors.

```
y_train = np.asarray(train_labels).astype("float32")
y_test = np.asarray(test_labels).astype("float32")
```

- Now the data is ready to be given to the neural network.

4.1.3 Building your model

- The input data is a vector, and the labels are scalars (1 or 0).
- This is probably the simplest problem we will see.
- A good model for this problem is just stacking Dense layers with ReLU activation.
- When building Dense layers, there are two important design choices:
  - How many layers to use?
  - How many units in each layer?
- In Chapter 5, we will learn general rules to help make these choices.

4.1.3 Building your model

- For now, let's follow the suggested structure:
  - Two hidden layers with 16 units each
  - A third layer that outputs the sentiment (positive/negative) as a scalar value

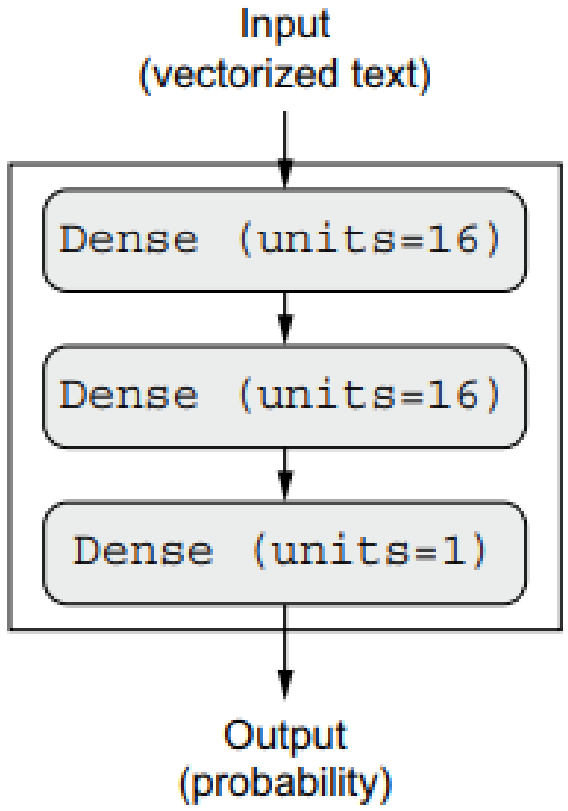


Figure 4.1 The three-layer model

4.1.3 Building your model

- Figure 4-1 shows this neural network.
- The next code is the Keras implementation of this network, similar to the Keras code we saw before with the MNIST example.

Listing 4.4 Model definition

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

4.1 Classifying movie reviews: A binary classification example

4.1.3 Building your model

Listing 4.4 Model definition

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

- The first parameter given to a Dense layer is the number of units, which is the dimension of the layer's representation space.
- In Chapters 2 and 3, we learned that a Dense layer with ReLU activation is built by connecting tensor operations like this.

```
output = relu(dot(input, W) + b)
```

4.1 Classifying movie reviews: A binary classification example

4.1.3 Building your model

- Having 16 units means the weight matrix W has the shape (input_dimension, 16).
- When we do a dot product between input data and W, the input is projected into a 16-dimensional space (then we add bias b and apply ReLU).
- The dimension of the representation space can be seen as the freedom the model has to learn hidden patterns.
- If we increase the number of units (make the space higher dimensional), the model can learn more complex patterns, but:
  - The computation cost becomes higher.
  - The model may also learn undesired patterns (good for training data, but not for test data).

4.1.3 Building your model

- The hidden layers use the ReLU activation function, and the last layer uses the sigmoid activation function to output a probability (a value between 0 and 1).
  - ➢ If the probability is close to 1, it means the review is likely **positive**.
- ReLU makes all negative values become 0 (see Figure 4-2).
- Sigmoid squeezes any value into the range [0, 1], so the output can be understood as a probability (see Figure 4-3).

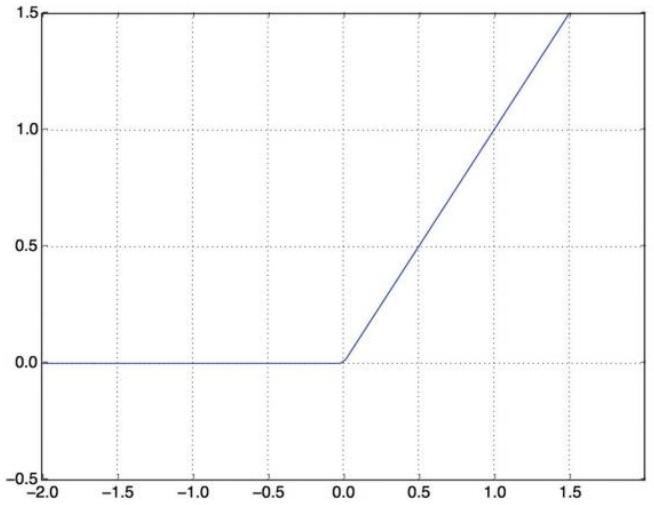


Figure 4.2 The rectified linear unit function

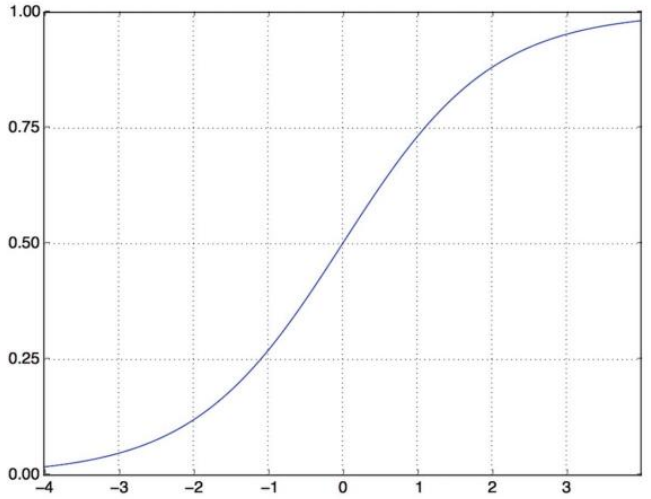


Figure 4.3 The sigmoid function

4.1 Classifying movie reviews: A binary classification example

4.1.3 Building your model

- What is an activation function? Why do we need it?
  - Without an activation function (also called non-linearity), a Dense layer only does two linear operations: dot product and addition. → `output = dot(input, W) + b`
  - This means the layer can only learn a linear transformation of the input.
  - Its hypothesis space is just all possible linear transformations of the input into, for example, a 16-dimensional space.
  - Such a space is very limited. Even if we stack many linear layers, it is still just one linear transformation.
  - (As we saw in Chapter 2) adding more linear layers does not expand the hypothesis space.
  - To make the hypothesis space richer and to get the benefit of deep networks, we need non-linearity (activation functions).
  - ReLU is the most popular activation function in deep learning.
  - There are also similar ones like PReLU, ELU, etc.

4.1 Classifying movie reviews: A binary classification example

4.1.3 Building your model

- Finally, we choose the loss function and the optimizer.
- Because this is a binary classification problem and the model output is a probability (a single unit with a sigmoid activation at the end), the right choice is binary_crossentropy loss.
- This is not the only option — we could also use mean_squared_error.
- But when the model outputs probabilities, crossentropy is the best choice.
- Crossentropy comes from information theory and measures the difference between two probability distributions.
- Here, it measures the difference between the true distribution and the predicted distribution.

4.1.3 Building your model

- The optimizer used is RMSprop.
- This optimizer is usually a good default choice for almost any problem.
- Next, we set up the model with the RMSprop optimizer and the binary_crossentropy loss function.
- During training, we use accuracy to monitor performance.

Listing 4.5 Compiling the model

```
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
```

4.1.4 Validating your approach

- A deep learning model must not be evaluated on the training data.
- The standard practice is to use a validation set to monitor accuracy during training.
- Here, we will take 10,000 samples from the training data to make a validation set.

Listing 4.6 Setting aside a validation set

```
x_val = x_train[:10000]
partial_x_train = x_train[10000:]
y_val = y_train[:10000]
partial_y_train = y_train[10000:]
```

- Even on a CPU, each epoch takes less than 2 seconds.
- The full training takes more than 20 seconds.
- At the end of each epoch, the model calculates loss and accuracy on the 10,000 validation samples, which adds a small delay.

4.1.4 Validating your approach

- The model.fit() method returns a History object.
- This object has a history attribute, which is a dictionary that stores all the information from the training process.

Listing 4.7 Training your model

```
history = model.fit(partial_x_train,
                    partial_y_train,
                    epochs=20,
                    batch_size=512,
                    validation_data=(x_val, y_val))
```

```
>>> history_dict = history.history
>>> history_dict.keys()
[u"accuracy", u"loss", u"val_accuracy", u"val_loss"]
```

4.1.4 Validating your approach

- This dictionary has 4 items, one for each metric monitored during training and validation.
- In the next two code examples, we will use Matplotlib to plot the loss (Figure 4-4) and accuracy (Figure 4-5) for both training and validation data.
- Because the model starts with random initialization, your results may look a little different from the book's results.

4.1 Classifying movie reviews: A binary classification example

▣ 4.1.4 Validating your approach

Listing 4.8 Plotting the training and validation loss

```
import matplotlib.pyplot as plt
history_dict = history.history
loss_values = history_dict["loss"]
val_loss_values = history_dict["val_loss"]
epochs = range(1, len(loss_values) + 1)
plt.plot(epochs, loss_values, "bo", label="Training loss")
plt.plot(epochs, val_loss_values, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

"bo" is for "blue dot."

"b" is for "solid blue line."

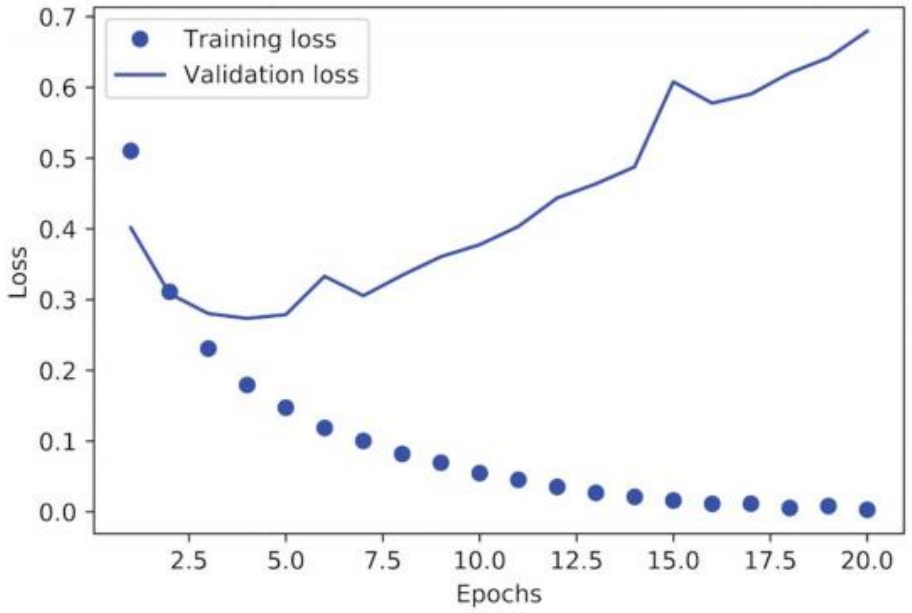


Figure 4.4 Training and validation loss

▣ 4.1.4 Validating your approach

Listing 4.9 Plotting the training and validation accuracy

```
plt.clf()                                   # Clears the figure
acc = history_dict["accuracy"]
val_acc = history_dict["val_accuracy"]
plt.plot(epochs, acc, "bo", label="Training acc")
plt.plot(epochs, val_acc, "b", label="Validation acc")
plt.title("Training and validation accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

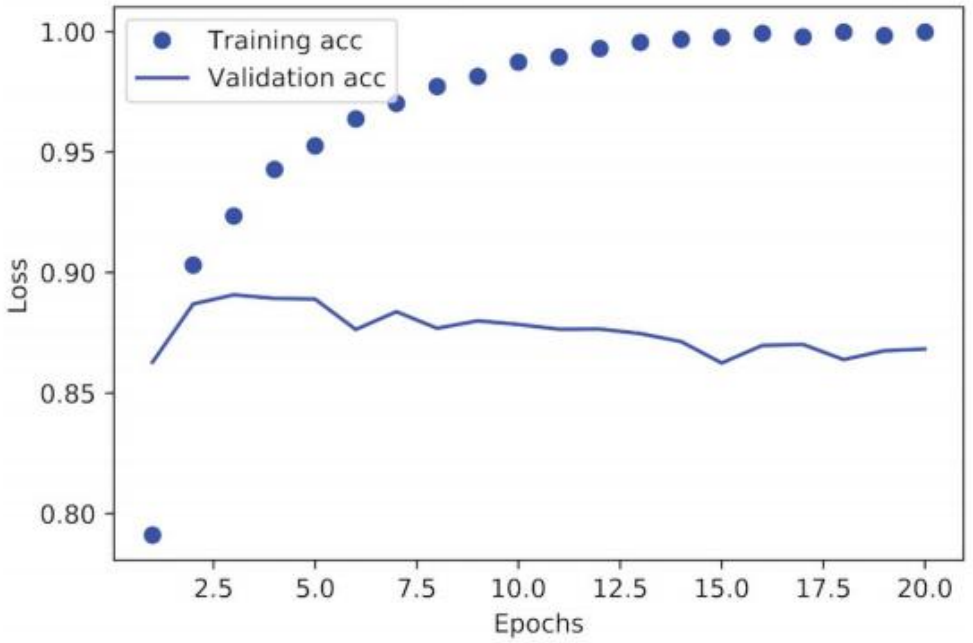


Figure 4.5 Training and validation accuracy

4.1 Classifying movie reviews: A binary classification example

4.1.4 Validating your approach

- As we can see here, the training loss goes down each epoch, and the training accuracy goes up each epoch.
- This is expected, because gradient descent always reduces the loss step by step.
- But the validation loss and accuracy do not follow the same trend.
- Around the 4th epoch, the graphs start to reverse.
- This is an example of the warning: a model that works well on training data may not work well on new data.
- The exact term for this is overfitting.
- After the 4th epoch, the model is too optimized for the training data and learns patterns that don't generalize to new data.
- To avoid this, we could stop training after the 4th epoch.
- In general, there are many techniques to reduce overfitting.

4.1.4 Validating your approach

- Let's train a **new neural network** again from the start, but this time for only **4 epochs**, and then test it on the **test data**.

Listing 4.10 Retraining a model from scratch

```
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),

    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(x_train, y_train, epochs=4, batch_size=512)
results = model.evaluate(x_test, y_test)
```

```
>>> results
[0.2929924130630493, 0.88327999999999995]
```

The first number, 0.29, is the test loss, and the second number, 0.88, is the test accuracy.

4.1.5 Using a trained model to generate predictions on new data

- After training the model, we want to use it in a real environment.
- With the predict() method, we can estimate the probability that a review is positive.

```
>>> model.predict(x_test)
array([[ 0.98006207]
       [ 0.99758697]
       [ 0.99975556]
       ...,
       [ 0.82167041]
       [ 0.02885115]
       [ 0.65371346]], dtype=float32)
```

- For some samples, the model is very confident (like 0.99 or higher, 0.01 or lower).
- For other samples, the model is less confident (like 0.6, 0.4).

4.1.6 Homework

- In the next experiments, we will see that the chosen structure can improve more, but it is already at a reasonable level.
  - Here, we used 2 hidden layers before the final classification layer.
  - Let's try with 1 or 3 hidden layers and check how it affects validation and test accuracy.
  - Try changing the number of units in the layers: 32 units, 64 units, etc.
  - Try using MSE loss instead of binary_crossentropy.
  - Try using tanh activation instead of ReLU (tanh was popular in early neural networks).

4.2 Regression

- The previous examples were considered classification problems, where the goal was to predict a single discrete label of an input data point.
- Another common type of machine learning problem is regression, which consists of predicting a continuous value instead of a discrete label:
  - for instance, predicting the temperature tomorrow, given meteorological data or predicting the time that a software project will take to complete, given its specifications.

4.2 Predicting house prices: A regression example

4.2.1 The Boston housing price dataset

- Let's predict the median house price given data such as crime rate, tax rate, etc. from the suburbs of Boston in the mid-1970s.
- The dataset here is different from the previous two examples.
- It has 506 data points:
  - **404 training samples**
  - **102 test samples**
- The scale of each feature (e.g., crime rate) is different.
  - Some values are ratios between **0 and 1**.
  - Some values range between **1 and 12** or **1 and 100**.

Listing 4.23 Loading the Boston housing dataset

```
from tensorflow.keras.datasets import boston_housing
(train_data, train_targets), (test_data, test_targets) = (
    boston_housing.load_data())
```

4.2 Predicting house prices: A regression example

4.2.1 The Boston housing price dataset

- As we can see, there are 404 training samples and 102 test samples.

```
>>> train_data.shape
(404, 13)
>>> test_data.shape
(102, 13)
```

- Each sample has 13 numerical features.
- Examples of features:
  - Crime rate per person
  - Average number of rooms per house
  - Access to highways

4.2 Predicting house prices: A regression example

4.2.2 Preparing the data

- Putting values with different scales directly into a neural network causes problems.
- The model might try to adjust automatically, but this makes learning harder.
- A common way to handle this is to normalize each feature.
- For each feature (each column in the input data), subtract the mean and divide by the standard deviation.
- This makes the feature's mean close to 0 and its standard deviation 1.
- With NumPy, this can be done easily.

Listing 4.24 Normalizing the data

```
mean = train_data.mean(axis=0)
train_data -= mean
std = train_data.std(axis=0)
train_data /= std
test_data -= mean
test_data /= std
```

4.2 Predicting house prices: A regression example

4.2.2 Preparing the data

- When normalizing the test data, remember to use the values calculated from the training data.
- In machine learning, you must never use values calculated from the test data.
- This rule applies even to simple steps like data normalization.

4.2 Predicting house prices: A regression example

4.2.3 Building your model

- Because the dataset is small, we use a small model with 2 hidden layers, each having 64 units.
- In general, the fewer training samples we have, the easier overfitting happens.
- Using a smaller model is one way to avoid overfitting.

Listing 4.25 Model definition

```
def build_model():
    model = keras.Sequential([
        layers.Dense(64, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(1)
    ])
    model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
    return model
```

Because we need to instantiate the same model multiple times, we use a function to construct it.

4.2 Predicting house prices: A regression example

4.2.3 Building your model

- The last layer of this model has one unit and no activation function (called a linear layer).
- This setup is typical for scalar regression (predicting one continuous value).
- If we add an activation function, it limits the range of the output.
- For example, if we use a sigmoid activation in the last layer, the model will only predict values between 0 and 1.
- Here, the last layer is purely linear, so the model can freely predict values in any range.

4.2 Predicting house prices: A regression example

4.2.3 Building your model

- This model is compiled with the MSE loss function.
- MSE stands for Mean Squared Error, which measures the squared distance between prediction and target.
- It is a loss function widely used in regression problems.
- During training, we also monitor a new metric: Mean Absolute Error (MAE).
- MAE is the absolute distance between prediction and target.
- For example, if MAE = 0.5 in this case, it means the prediction is off by about $500 on average.

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- To evaluate the model, we split the data into training set and validation set, just like in the earlier example.
- Because there are not many data points, the validation set becomes very small (about 100 samples).
- Depending on which points are chosen, the validation score can change a lot.
- This means the variance of validation scores is high.
- As a result, we cannot evaluate the model reliably.
- In this case, the best way is to use K-fold cross-validation (see Figure 4-8).

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- Split the data into K parts (folds) (usually K = 4 or 5).
- Build K models. For each model, train on K-1 folds and test on the remaining fold.
- The model's validation score is the average of the K scores.
- It is easier to understand when you see it in code.

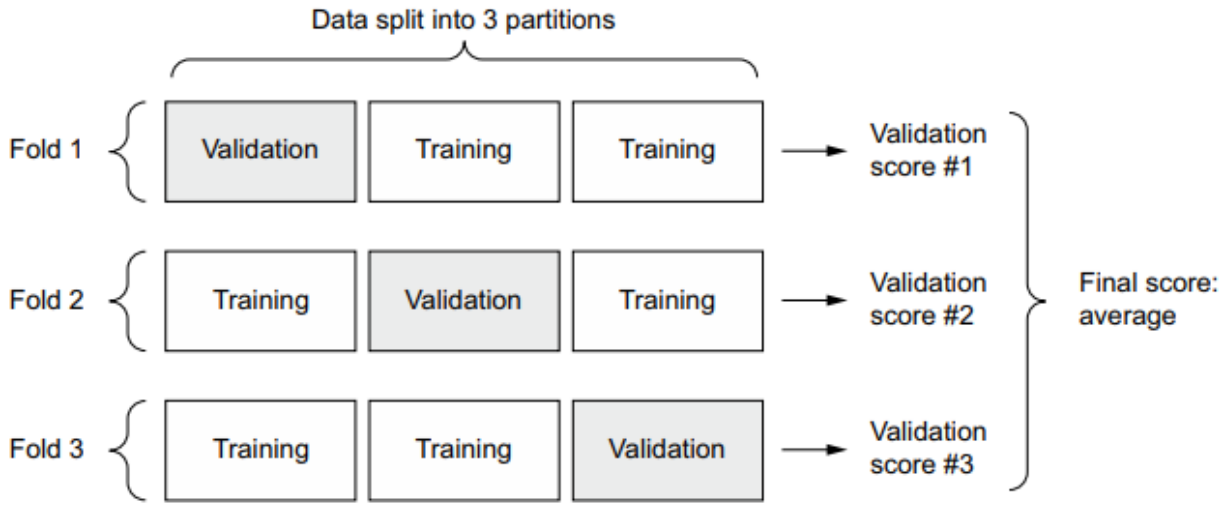


Figure 4.8 K-fold cross-validation with K=3

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

Listing 4.26 K-fold validation

```
k = 4
num_val_samples = len(train_data) // k
num_epochs = 100
all_scores = []
for i in range(k):
    print(f"Processing fold #{i}")
    val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
    val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]
    partial_train_data = np.concatenate(
        [train_data[:i * num_val_samples],
         train_data[(i + 1) * num_val_samples:]],
        axis=0)
    partial_train_targets = np.concatenate(
        [train_targets[:i * num_val_samples],
         train_targets[(i + 1) * num_val_samples:]],
        axis=0)
    model = build_model()
    model.fit(partial_train_data, partial_train_targets,
              epochs=num_epochs, batch_size=16, verbose=0)
    val_mse, val_mae = model.evaluate(val_data, val_targets, verbose=0)
    all_scores.append(val_mae)
```

Prepares the validation data: data from partition #k

Prepares the training data: data from all other partitions

Builds the Keras model (already compiled)

Trains the model (in silent mode, verbose = 0)

Evaluates the model on the validation data

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- Running with num_epochs = 100 gives the following results.

```
>>> all_scores
[2.112449, 3.0801501, 2.6483836, 2.4275346]
>>> np.mean(all_scores)
2.5671294
```

- Validation scores change a lot (from 2.1 to 3.1) depending on the validation set.
- The average score (2.6) is more reliable than each single score.
- This is the key idea of K-fold cross-validation.
- In this example, the average error is about $2,600.
- Since house prices range between $10,000 and $50,000, this error is still quite large.

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- Let's train this model for longer, about 500 epochs.
- To record how much the model improves at each epoch, we slightly change the training loop.
- At every epoch, we save the validation score of each fold in the log.

Listing 4.27 Saving the validation logs at each fold

```
num_epochs = 500
all_mae_histories = []
for i in range(k):
    print(f"Processing fold #{i}")
    val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
    val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]
    partial_train_data = np.concatenate(
        [train_data[:i * num_val_samples],
         train_data[(i + 1) * num_val_samples:]],
        axis=0)
    partial_train_targets = np.concatenate(
        [train_targets[:i * num_val_samples],
         train_targets[(i + 1) * num_val_samples:]],
        axis=0)
    model = build_model()
    history = model.fit(partial_train_data, partial_train_targets,
                        validation_data=(val_data, val_targets),
                        epochs=num_epochs, batch_size=16, verbose=0)
    mae_history = history.history["val_mae"]
    all_mae_histories.append(mae_history)
```

Prepares the validation data: data from partition #k

Prepares the training data: data from all other partitions

Builds the Keras model (already compiled)

Trains the model (in silent mode, verbose=0)

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- Then, calculate the average MAE score for each epoch across all folds.

Listing 4.28 Building the history of successive mean K-fold validation scores

```
average_mae_history = [
    np.mean([x[i] for x in all_mae_histories]) for i in range(num_epochs)]
```

- If we draw it as a graph, it will look like Figure 4-9.

Listing 4.29 Plotting validation scores

```
plt.plot(range(1, len(average_mae_history) + 1), average_mae_history)
plt.xlabel("Epochs")
plt.ylabel("Validation MAE")
plt.show()
```

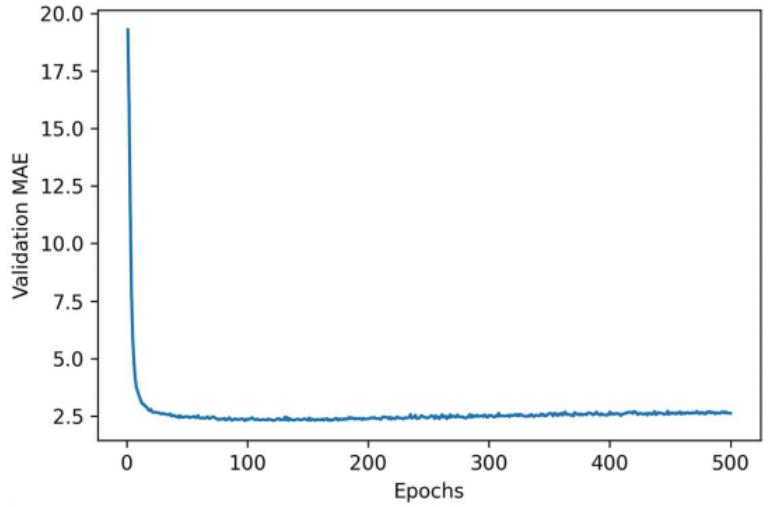


Figure 4.9 Validation MAE by epoch

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- This graph is hard to read because the range is too large.
- In the first few epochs, the MAE is much higher than in later epochs.
- To make the rest of the curve easier to see, let's remove the first 10 data points (which are on a very different scale).

Listing 4.31 Training the final model

```
model = build_model()                    # Gets a fresh, compiled model
model.fit(train_data, train_targets,      # Trains it on the entirety of the data
          epochs=130, batch_size=16, verbose=0)
test_mse_score, test_mae_score = model.evaluate(test_data, test_targets)
```

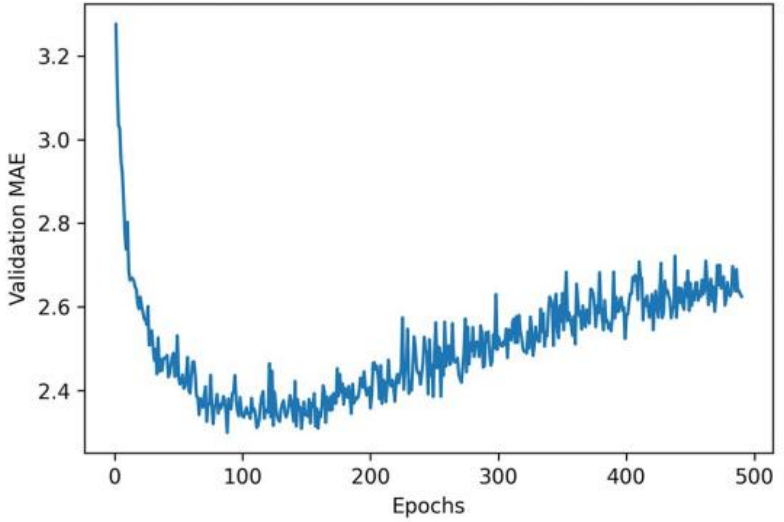


Figure 4.10 Validation MAE by epoch, excluding the first 10 data points

4.2 Predicting house prices: A regression example

4.2.4 Validating your approach using K-fold validation

- As shown in Figure 4-10, the validation MAE stopped decreasing after around the 120th–140th epoch (including the first ten epochs omitted from the graph).
- After this point, overfitting begins.
- Once tuning of various model parameters is complete (not only the number of epochs but also the size of hidden layers can be adjusted), train the final model using all the training data with the best parameters.
- Then, evaluate the performance on the test data.

```
>>> test_mae_score
2.4642276763916016
```

4.2 Predicting house prices: A regression example

4.2.5 Generating predictions on new data

- In the binary classification model, when calling the predict() method, it returns a scalar score between 0 and 1 for each sample.
- In the multi-class model, each sample gets a probability distribution over all classes.
- Now, with this scalar regression model, the predict() method can be used to estimate the price of a new sample in units of $1,000.

```
>>> predictions = model.predict(test_data)
>>> predictions[0]
array([9.990133], dtype=float32)
```

- For example, the price of the first house in the test set is predicted to be about $10,000.

4.3 Summary

- **The most common machine learning tasks using vector data are binary classification and scalar regression.**
  - In the “Summary” section of this chapter, the key lessons from these types of tasks have been organized.
  - The loss functions and evaluation metrics used in regression differ from those in classification.
  - Typically, raw data must be preprocessed before being fed into the neural network.
  - If the features of the data have different ranges, each feature should be independently scaled during preprocessing.
  - As training progresses, the neural network may begin to overfit, leading to poor results on new data.
  - When training data is limited, models with only one or two hidden layers are used to avoid overfitting.
  - If the data is divided into many categories, hidden layers that are too small can cause an information bottleneck.
  - When the dataset is small, K-fold cross-validation helps provide a reliable evaluation of the model.



Q & A

Prof. Chang-Hee Han